TIL

エージェント内部構造

LLM-Agent-Controller

[概要]

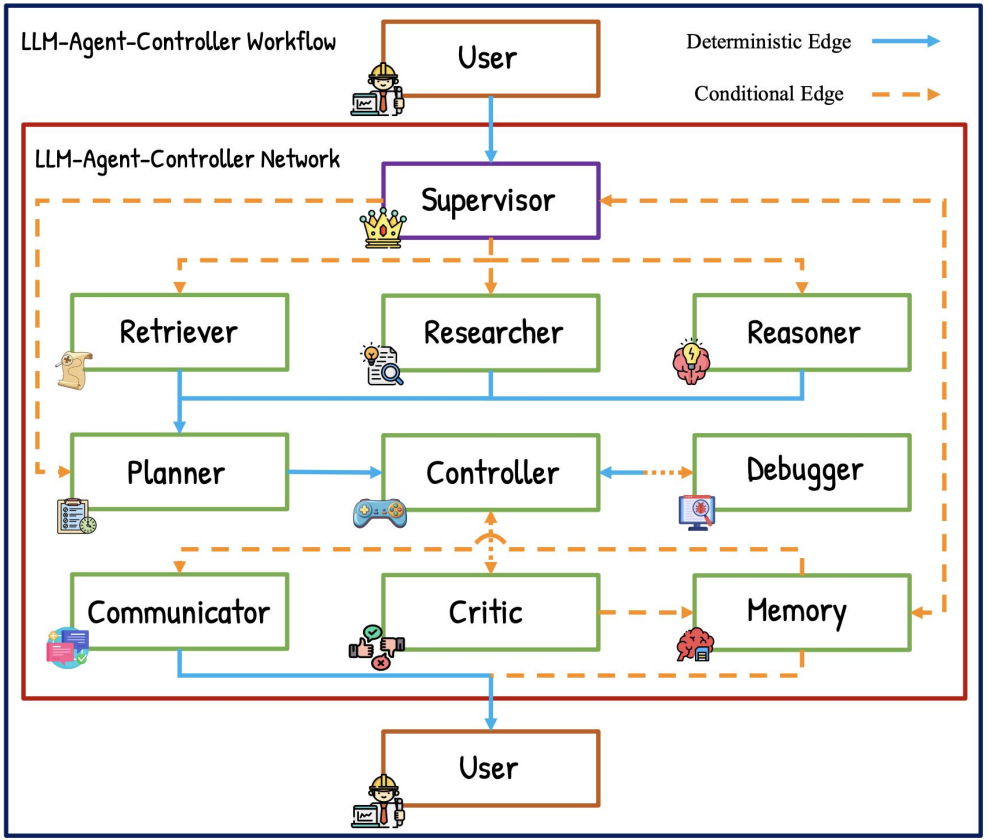
- 本システムは、中央のスーパーバイザー（Supervisor）の管理下で、その配下の専門エージェント（Retriever Agent、Planner Agent、Memory Agent、etc）が協調して様々なタスクを処理する。
- 専門エージェントの中核として、Controller Agent が制御工学のタスクを扱う。

LLM-Agent-Controller

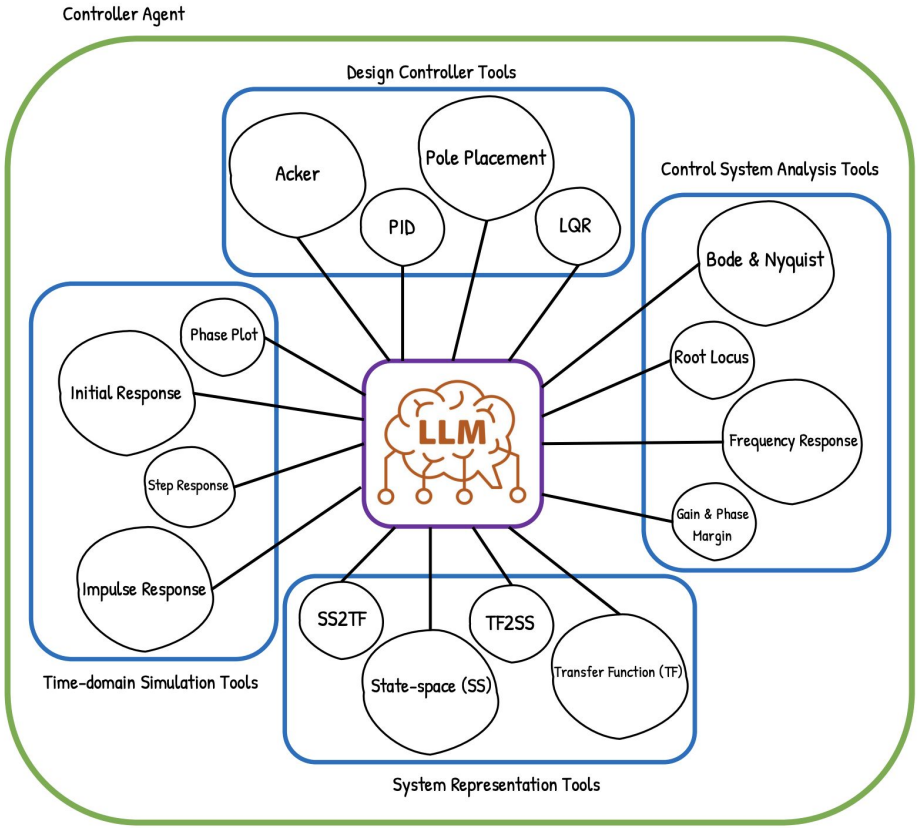
[手法]

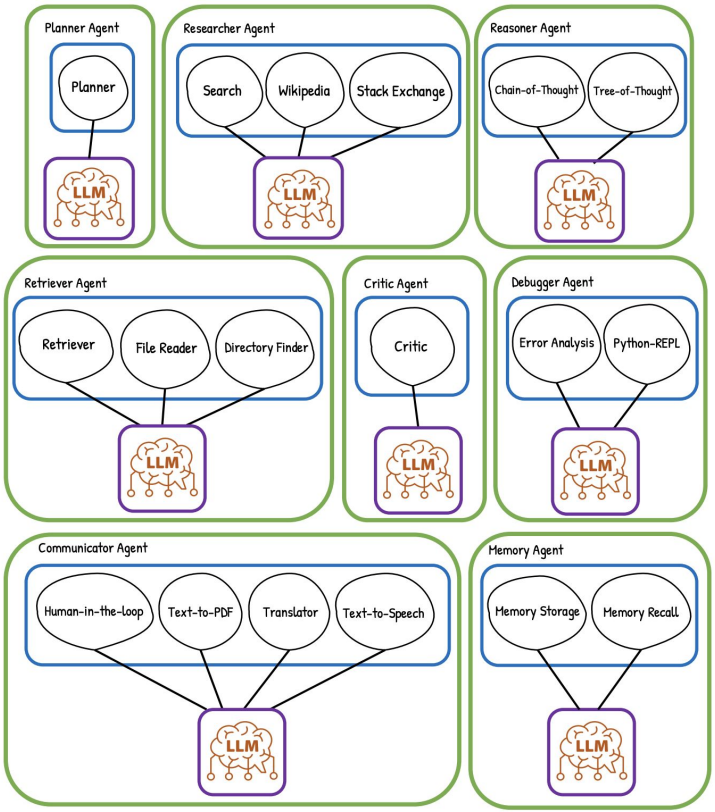
- ユーザーからの問い合わせを受信後、スーパーバイザーが高レベルの意思決定を行い、配下の専門エージェントを呼び出すかどうかを判断する。
- 情報を取得するエージェントとしては、Retriever Agent（RAGによる外部ソースからのデータ取得）や Research Agent（インターネット検索）がある。
- Planner Agent は、Controller Agent が使用するべきツールの実行順序を整理する。
- Controller Agent は、制御工学に関連した140種類以上のツールを備えている。
- Memory Agent は、短期メモリ及び長期メモリを効率的に管理する。

LLM-Agent-Controller



LLM-Agent-Controller





Cognitive Architectures for Language Agents

[概要]

- 汎用的なLLMエージェントを設計するための概念的枠組み「Cognitive Architecture for Language Agents（CoALA）」の提案。
- 20世紀前半には、数学分野をはじめとして、プロダクションシステムを導入する潮流があった。プロダクションシステムでは、規則を「前提条件」と「その条件が満たされた時に実行する行動」のペアとして定義する。
- 従来AIでは、あらかじめ多くの規則を定義する必要があった。それに対しLLMは、与えられた入力系列に対して、どのような出力系列が最適かという確率分布を学習しており、確率的なプロダクションシステムとして捉えることができる。
- 本論文では、LLMとプロダクションシステムの類似性に着目し、プロダクションシステムに用いられてきた認知アーキテクチャをLLMエージェントに適用する枠組みとして、CoALAを提案する。

Cognitive Architectures for Language Agents

$$
\begin{aligned}
(\text{temperature} > 70^\circ) \wedge (\text{temperature} < 72^\circ) &\rightarrow \text{stop} \\
\text{temperature} < 32^\circ &\rightarrow \text{call for repairs; turn on electric heater} \\
(\text{temperature} < 70^\circ) \wedge (\text{furnace off}) &\rightarrow \text{turn on furnace} \\
(\text{temperature} > 72^\circ) \wedge (\text{furnace on}) &\rightarrow \text{turn off furnace}
\end{aligned}
$$

Cognitive Architectures for Language Agents

[手法]

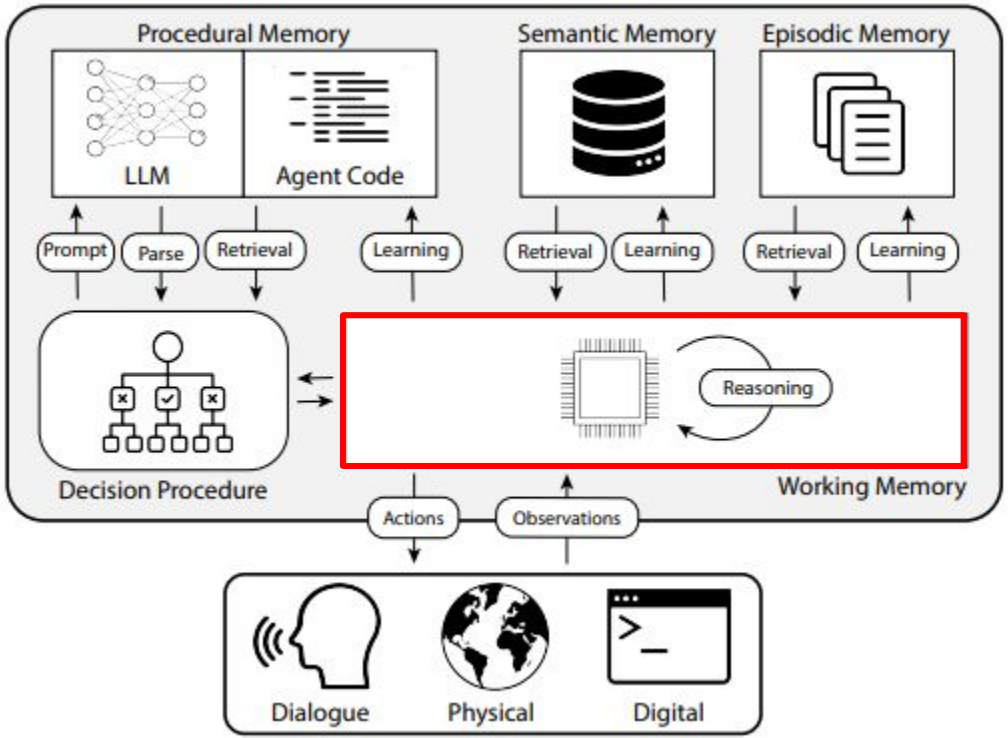
- CoALAは、LLMエージェントを主に3つのコンポーネントに整理する。
  ① メモリ機能（短期記憶・長期記憶）
  ② 行動空間（内部行動・外部行動）
  ③ 意思決定サイクル
- CoALAを用いて特定アプリケーション向けのLLMエージェントを開発するには、上記3つのコンポーネントを設計する必要がある。

Cognitive Architectures for Language Agents

[手法 - メモリ機能（短期記憶）]

- 論文中では、ワーキングメモリと記載されている。
- 現在の意思決定サイクルにおいて、LLMエージェントの状態を保持する。
  これには、最近の知覚入力、推論によって生成された知識、長期記憶から検索された知識、前回の意思決定サイクルから引き継がれた中核的な情報（LLMエージェントの目標など）が含まれる。
- 短期記憶の情報は、LLMに入力される。
- 短期記憶はLLMと長期記憶を繋ぐインタフェースとしても機能する。LLMエージェントの各構成要素を結びつける中心的なハブともいえる。

Cognitive Architectures for Language Agents



Cognitive Architectures for Language Agents

[手法 - メモリ機能（長期記憶）]

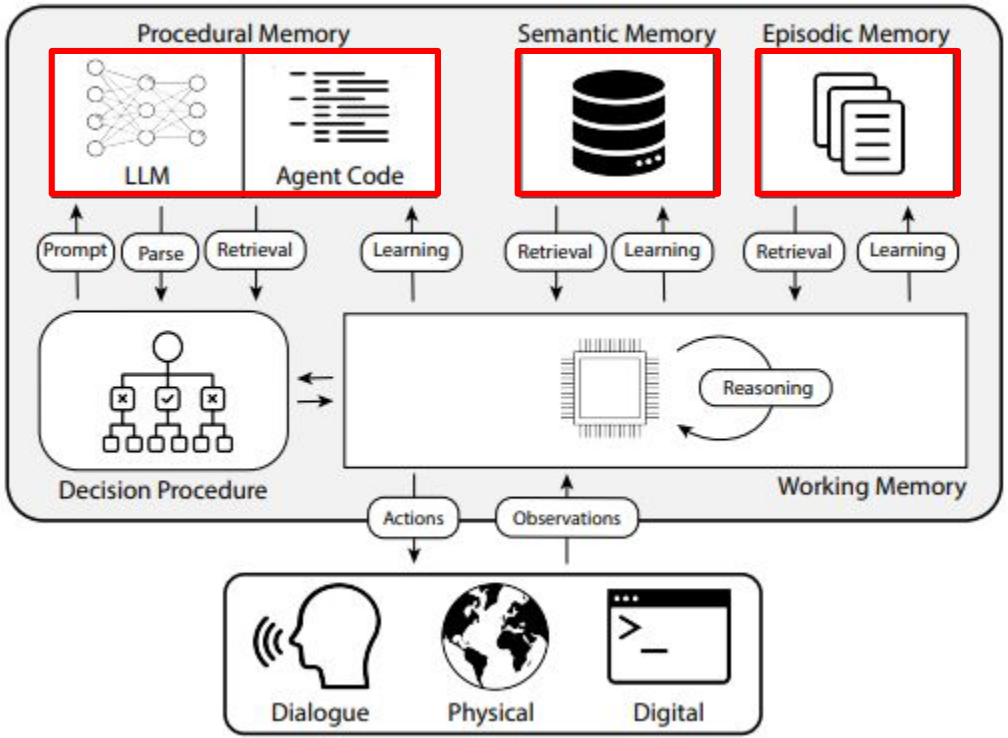
- 次の3種類に分けられる。
- エピソード記憶（Episodic Memory）:
  過去の意思決定サイクルにおける、LLMエージェントの行動系列を保持する。意思決定サイクルの計画段階では、エピソード記憶を短期記憶に抽出し、LLMの推論を支援することがある。
- 意味記憶（Semantic Memory）:
  世界に関する事実を保持する。

Cognitive Architectures for Language Agents

[手法 - メモリ機能（長期記憶）]

- 手続き記憶（Procedural Memory）：
  LLMの重みに保持された暗黙的知識（Implicit Knowledge）と、LLMエージェントのコードに記述された明示的知識（Explicit Knowledge）からなる。
  LLMの重み更新（ファインチューニング）は高コストであるため、LLMエージェントが稼働中に自律的に更新するのは現状では難しい。
  また、LLMエージェント自身がソースコードを更新することも可能だが、エピソード記憶や意味記憶への書き込みに比べて遥かにリスクが高い。バグの混入や、設計者の意図に反する挙動を招く恐れがある。

Cognitive Architectures for Language Agents

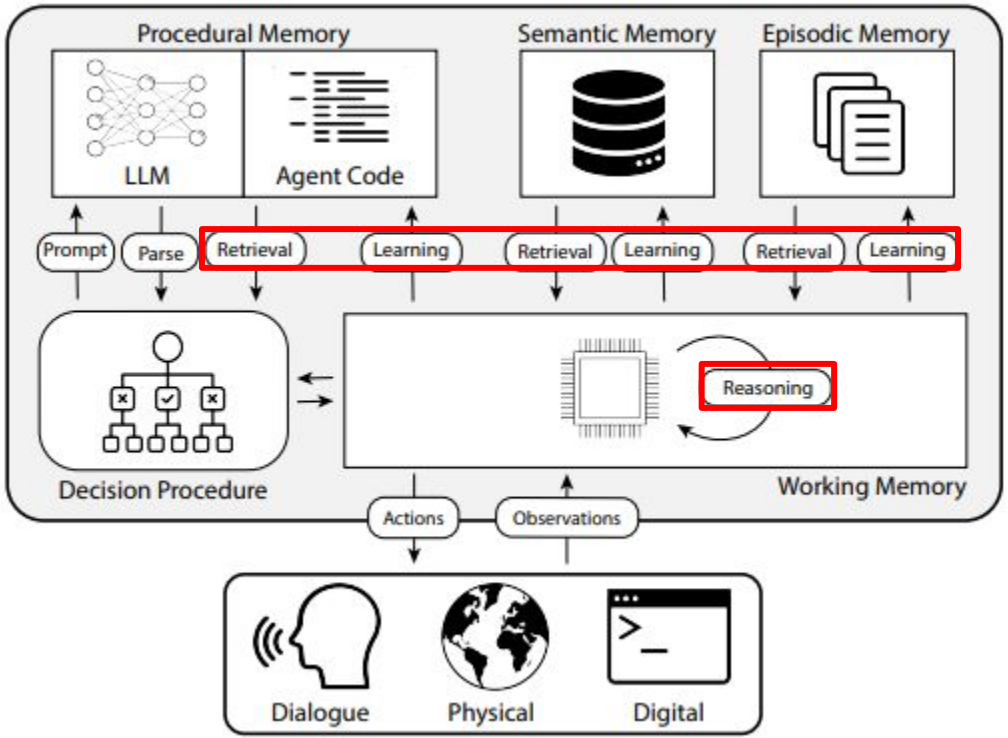


Cognitive Architectures for Language Agents

[手法 - 行動空間（内部行動）]

- 次の3種類に分けられる。なお、LLMエージェントを設計する際には、各メモリへのアクセス権限（読み取り・書き込み）をどう設定するかを決める必要がある。
- 検索：
  長期記憶から情報を短期記憶に読み出す。
- 推論：
  短期記憶の内容を処理し、新たな情報を生成する。短期記憶への書き込みも同時に行う。
- 学習：
  情報を長期記憶へ書き込む。

Cognitive Architectures for Language Agents

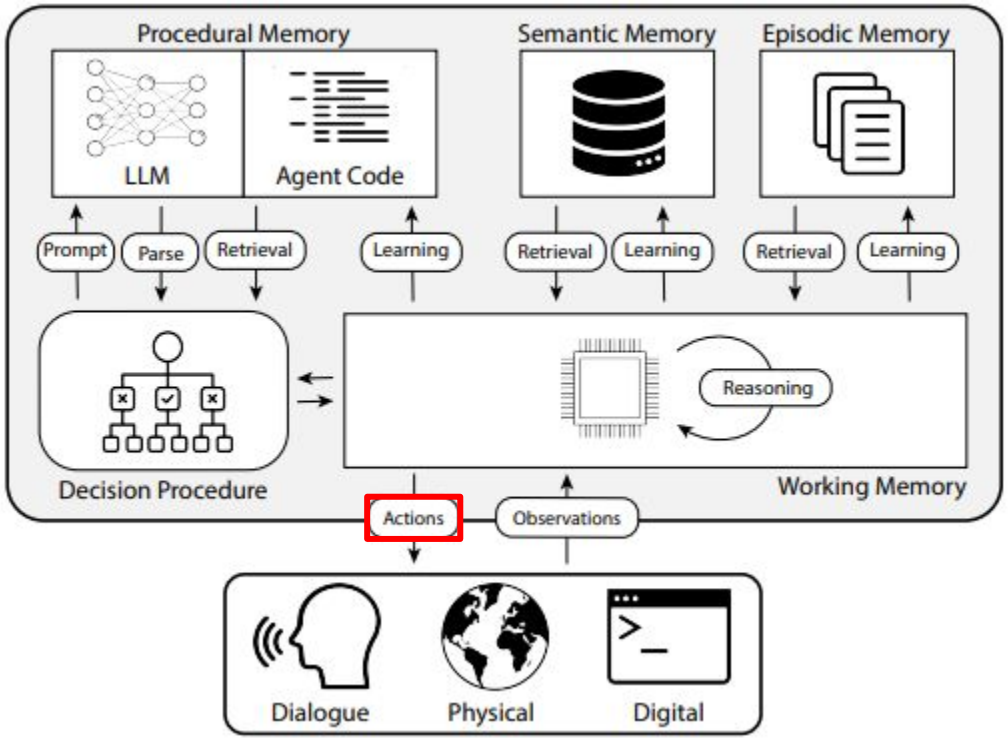


Cognitive Architectures for Language Agents

[手法 - 行動空間（外部行動）]

- 物理環境での行動に加え、デジタル環境におけるAPI呼び出し（検索、翻訳などのツール利用）も含まれる。

Cognitive Architectures for Language Agents

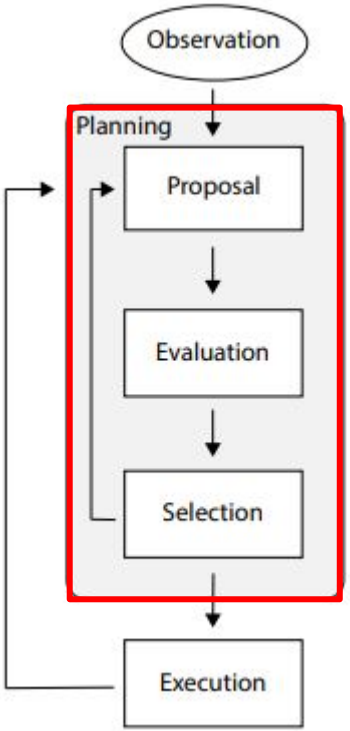


Cognitive Architectures for Language Agents

[手法 - 意思決定サイクル（計画段階）]

- 次の3種類に分けられる。推論や検索などの内部行動を柔軟に組み合わせながら、行動候補を生成・評価する。
- 提案：
  LLMが1つ以上の行動候補を生成する。通常は推論を用い、必要に応じて検索も行う。より高度なLLMエージェントでは、if-else や while-if のコードを用いる場合もある。
- 評価：
  各行動候補を定量的に評価する。この評価には、経験則的なルール⊥LLMの評価指標（perplexity）、LLMによる推論などが用いられる。
- 選択：
  行動候補から1つを選択するか、全てを棄却して再び提案ステップに戻るかを決定する。

Cognitive Architectures for Language Agents

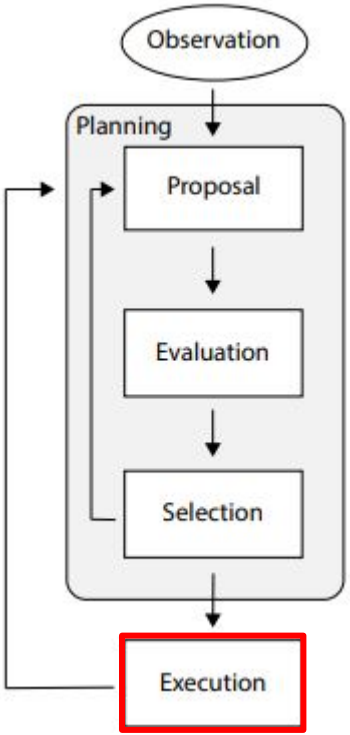


Cognitive Architectures for Language Agents

[手法 - 意思決定サイクル（実行段階）]

- 計画段階で選択された行動を実行する。
- 内部行動（学習）、あるいは外部行動（API呼び出しなど）を行う。

Cognitive Architectures for Language Agents

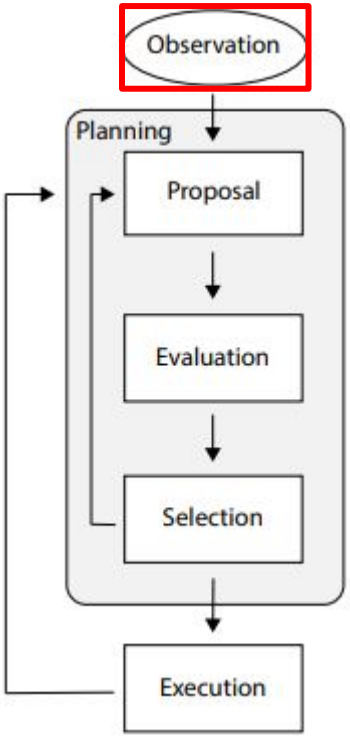


Cognitive Architectures for Language Agents

[手法 - 意思決定サイクル（観測段階）]

- 行動実行後、行動結果に対するフィードバックが与えられたうえで、次の意思決定サイクルへ進む。

Cognitive Architectures for Language Agents



claude-code

[概要 - KAIROS]

- assistant/sessionHistory.ts には、KAIROSと呼ばれるモードの実装がある。
- これは常駐するClaudeアシスタントで、ユーザーの入力なしに、監視やログファイルへの記録を行い、AIアシスタントが気付いた事柄に対して積極的に対応する。
- Ambient Agent の考え方に非常に似ている。
- 本システムには15秒の許容時間（timeout）が設定されている。AIアシスタントの介入が許容時間を超過するような操作については、全てpendingされる仕組みとなっている。
- AIアシスタントの応答を簡潔にし、メッセージでターミナルを占有しないようにするためのBriefモードも有効可能である。

claude-code

[概要 - ULTRAPLAN]

- 複雑な計画タスクをCloud Container Runtime (CCR) セッションにオフロードし、最大30分間の思考時間を与え、ブラウザから結果を承認できるようにするモード。
- 基本的なフローは以下の通り:
  1. Claude Code が、深い思考を要するタスクを特定する。
  2. 設定ファイルを介して、CCRセッションを起動する。
  3. 3秒間ごとに、ターミナルに思考状況を表示する。
     ブラウザベースのUIでは、計画の進捗状況を確認したり、結果を承認／却下することができる。
  4. 結果が承認されると、ターミナルに結果が転送される。

claude-code

※：プロンプト抜粋（原文は英語）

> 「あなたは夢を見ているのです。記憶のファイルをじっくりと見直す作業です。最近学んだことを、永続的で整理された記憶として統合することで、今後の学習セッションで素早く方向転換できるようになります。」

[概要 - autoDream]

- SubAgentとして動作するメモリ統合バックエンド。
- 以下条件を""全て""満たした場合に実行される：
  1. Time Gate：最後の動作から24時間経過している場合
  2. Session Gate：直近の動作から少なくとも5セッション経過している場合
  3. Lock Gate：（よく分らんかった）
- 実行後、autoDreamは事前定義されたプロンプト※（services/autoDream/consolidationPrompt.ts）に基づき、4つのフェーズを実施する。
  1. Memory.mdファイルから、内容を改善するべき既存トピックを特定する。
  2. 保持する価値のある新たな情報を探し出す。
  3. Memory.mdファイルを更新する。
  4. Memory.mdファイルを、200行以下 &25KB以下の状態にプルーニングする。

claude-code

[概要 - Coordinator Mode]

- coordinator/coordinatorMode.ts を介して起動されるマルチエージェントシステム。
- 機能フラグ（COORDINATOR_MODE）が有効になると、
  Claude Codeは単一エージェントから、複数のワーカーエージェントを並列に生成、指示（作業の続行及びストップ）するコーディネーターエージェントに変化する。
- ワーカーの生成及び指示は、コーディネーターのツールとして実装されている。

claude-code

※：システムプロンプトで250行程ある。詳細はソースを参照

[概要 - Coordinator Mode]

- コーディネーターに与えるシステムプロンプトは以下の通り※：
  1. コーディネーターの役割
  2. ツールの情報及びXMLメッセージフォーマットの説明
  3. タスクのワークフロー（Phases）の例示
  4. ワーカーに与えるプロンプトを作成する際の注意点（例示あり）
  5. やり取りしている様子の例示

段階	誰が	目的
研究	労働者（並列）	コードベースを調査し、ファイルを見つけ、問題を理解する
合成	コーディネーター	調査結果を読み、問題点を理解し、仕様書を作成する
実装	労働者	仕様ごとに的を絞った変更を行い、コミットする
検証	労働者	変更をテストすると正常に動作します

claude-code

[概要 - Coordinator Mode]

- ワーカーエージェント同士はXMLメッセージ（task-notification）を介して通信することができ、永続的な知識を共有するためのディレクトリ（shared scratchpad directory）もある。
- 本システムには、コンテキスト隔離のためのAgent Teams／Swarm機能や、メモリ同期機能などもある。

claude-code

[概要 - System Prompt Architecture]

- システムプロンプトは、事前定義された単一の文字列ではなく、キャッシュされたモジュール（constants/systemPromptSections.ts 等に実装あり）から実行時に構成する手法を取っている。
- システムプロンプトの各モジュールは、SYSTEM_PROMPT_DYNAMIC_BOUNDARYマーカーにより2種類に分割される。
  - 静的モジュール：ユーザーを跨いでキャッシュされる、不変のシステムプロンプト
  - 動的モジュール：あるユーザー（もしくはセッション）ごとにキャッシュされる、特有なシステムプロンプト。

claude-code

[概要 - Tools]

- tools/ では、40以上のツールが定義されている。
- getAllBaseTools()を介してツールが登録された後、ユーザー属性や環境フラグ、フォルダ権限設定などによってフィルタリングされる。

Tool	What It Does
AgentTool	Spawn child agents/subagents
BashTool / PowerShellTool	Shell execution (with optional sandboxing)
FileReadTool / FileEditTool / FileWriteTool	File operations
GlobTool / GrepTool	File search (uses native bfs / ugrep when available)
WebFetchTool / WebSearchTool / WebBrowserTool	Web access
NotebookEditTool	Jupyter notebook editing
SkillTool	Invoke user-defined skills
REPLTool	Interactive VM shell (bare mode)
LSPTool	Language Server Protocol communication
AskUserQuestionTool	Prompt user for input
EnterPlanModeTool / ExitPlanModeV2Tool	Plan mode control
BriefTool	Upload/summarize files to claude.ai
SendMessageTool / TeamCreateTool / TeamDeleteTool	Agent swarm management
TaskCreateTool / TaskGetTool / TaskListTool / TaskUpdateTool / TaskOutputTool / TaskStopTool	Background task management
TodoWriteTool	Write todos (legacy)
ListMcpResourcesTool / ReadMcpResourceTool	MCP resource access
SleepTool	Async delays
SnipTool	History snippet extraction
ToolSearchTool	Tool discovery
ListPeersTool	List peer agents (UDS inbox)
MonitorTool	Monitor MCP servers
EnterWorktreeTool / ExitWorktreeTool	Git worktree management
ScheduleCronTool	Schedule cron jobs
RemoteTriggerTool	Trigger remote agents
WorkflowTool	Execute workflow scripts
ConfigTool	Modify settings (internal only)
TungstenTool	Advanced features (internal only)
SendUserFile / PushNotification / SubscribePR	KAIROS-exclusive tools

claude-code

[概要 - compile-time feature flags]

- コンパイル時の機能フラグのONOFFにより、Claude Codeの各種機能の利用可否を設定している。(compile-time feature elimination)

Flag	What It Gates
`PROACTIVE` / `KAIROS`	Always-on assistant mode
`KAIROS_BRIEF`	Brief command
`BRIDGE_MODE`	Remote control via claude.ai
`DAEMON`	Background daemon mode
`VOICE_MODE`	Voice input
`WORKFLOW_SCRIPTS`	Workflow automation
`COORDINATOR_MODE`	Multi-agent orchestration
`TRANSCRIPT_CLASSIFIER`	AFK mode (ML auto-approval)
`BUDDY`	Companion pet system
`NATIVE_CLIENT_ATTESTATION`	Client attestation
`HISTORY_SNIP`	History snipping
`EXPERIMENTAL_SKILL_SEARCH`	Skill discovery

エージェント形態

AgentNet

[概要]

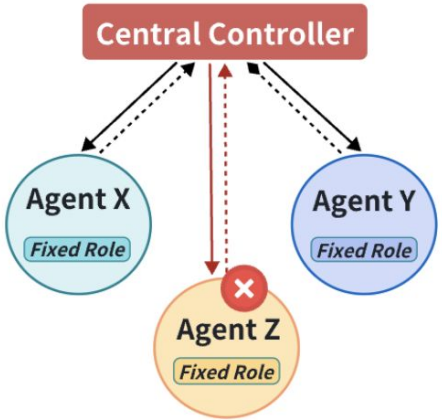
- 現在の主流である、マルチエージェントシステム（Multi-Agent System; MAS）の中央集権型アーキテクチャには、以下の課題がある。
  - 中央のコントローラーが単一障害点（Single Point of Failure; SPOF）となる。
  - エージェントの役割及びワークフローが静的であるため、タスク要件の複雑さに応じて動的にタスクを割り当てることができず、適応性に欠ける。
- 上記の課題を受け、本論文では、以下の特徴を持つ分散型アーキテクチャ「AgentNet」を提案する。
  - 単一障害点であったコントローラーを排除し、意思決定権を全てのエージェントに分散することで、フォールトトレランスが高まる。
  - RAGベースの記憶システムにより、過去の経験をもとに適応的に役割（専門性）を学習することが可能。また、リアルタイムに最適化されるネットワークトポロジーにより、ワークフローを柔軟に最適化できる。

AgentNet

Pre-Defined vs Self-Evolving Multi-Agent System

Pre-Defined Agents

hierarchical, static, single point of failure

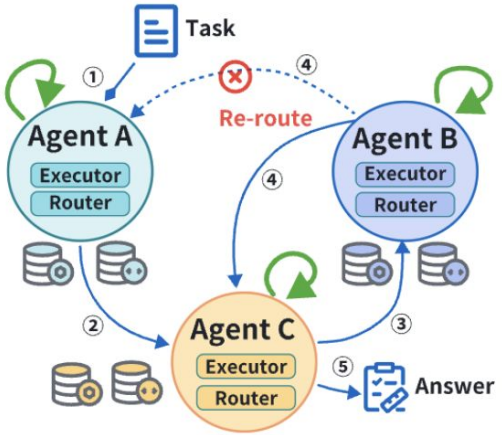


Key Features:

- Static role assignments
- Centralized task management
- Rigid workflows
- Single point of failure risk

Self-Evolving Agents (AgentNet)

Adaptive, decentralized, fault-tolerant



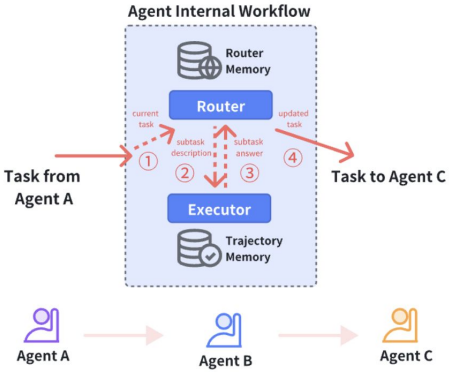
Key Features:

- Dynamic expertise development
- Autonomous task routing
- Experience-driven adaption
- Decentralized coordination

AgentNet

[手法]

- AgentNetの各エージェントは、ReAct（Reasoning and Acting）フレームワークに基づいており、主に2つのモジュールを内包している。
  - ルーター：クエリとコンテキスト（過去の経験）から、適切なアクションをLMが推論する。
  - エクゼキューター：ルーターでの推論結果及びコンテキスト（過去の経験）からLMが具体的なアクションに変換する。



AgentNet

[結果]

- 数学タスク、論理的質問応答タスク、及びAPI呼び出しタスクにおける正答率（Accuracy）の結果。
- MetaGPTなどの中央集権型MASを一貫して上回っている。

Backbone	Category	Method	MATH(Acc/%)	BBH(Acc/%)	API-Bank(Acc/%)
DeepSeek-V3	Single Agent	Direct	47.86	69.00	26.00
		React	77.14	88.00	29.00
		Synapse	89.28	92.00	28.00
		Self-Consistency	88.00	85.00	29.00
		Self-Refinement	87.14	84.00	25.00
	Multi-Agent	MorghAgent	39.29	56.00	16.00
		MetaGPT	92.14	64.00	22.00
		AFLOW	91.67	88.00	28.00
		GPTSwarm	72.14	90.00	21.00
		AgentNet	**92.86**	**94.00**	**30.00**
GPT-4o-mini	Single Agent	Direct	31.43	59.00	15.00
		React	55.71	80.00	24.00
		Synapse	77.14	79.00	22.00
		Self-Consistency	54.28	85.00	22.00
		Self-Refinement	68.57	81.00	23.00
	Multi-Agent	MorghAgent	80.71	56.00	16.00
		MetaGPT	73.57	53.00	19.00
		AFLOW	**85.00**	75.00	21.00
		GPTSwarm	**85.00**	**86.00**	13.00
		AgentNet	**85.00**	**86.00**	**29.00**
Qwen-turbo	Single Agent	Direct	37.85	57.00	27.00
		React	53.57	69.00	23.00
		Synapse	67.14	68.00	24.00
		Self-Consistency	64.28	70.00	28.00
		Self-Refinement	76.43	74.00	23.00
	Multi-Agent	MorghAgent	16.43	56.00	9.00
		MetaGPT	63.57	51.00	20.00
		AFLOW	**82.14**	57.00	22.00
		GPTSwarm	79.29	75.00	30.00
		AgentNet	81.43	**92.00**	**32.00**

AgentNet

[結果]

- エージェントの多様性がパフォーマンスに与える影響を調査した結果。
- 3エージェントの場合、LLMモデルと能力が同一（Fully Homogeneous）が最良のパフォーマンスを示した。小規模チームでは、均一な性能を持つエージェント同士の協調がより効果的であることを示唆している。
- 5エージェントの場合、LLMモデルと能力が同一（Fully Homogeneous）が最低のパフォーマンスを示した。大規模チームでは、LLMモデルまたは能力の多様性により、エージェントの協調性や相補的な推論能力が強化されることを示している。

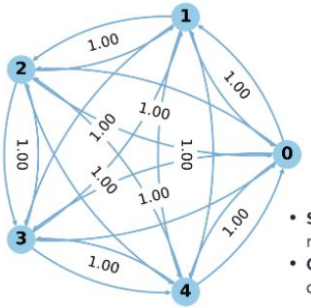
Setting	Fully Homogeneous	Skill Hetero.	LLM Hetero.	Both Hetero.
3 Agents	0.86	0.84	0.81	0.81
5 Agents	0.79	0.86	0.85	0.85

AgentNet

[結果]

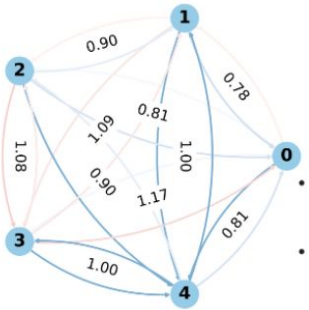
- ネットワークトポロジーの進化過程。
- 初期状態（左図）では、ネットワークは全結合状態にある。この段階ではエージェントは専門家しておらず、全てのエージェントがタスクを処理する能力を等しく備えている。
- 中間状態（中央図）では、ネットワークが進化（重みが変動）するとともに、エージェントが専門家を始めている。重みが増している部分は、より緊密な連携を示している。
- 最終状態（右図）までに、ネットワークは最適化され、エージェントは明確な専門性を持っていることが分かる。

AgentNet



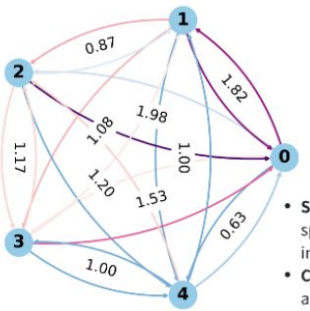
Start

- **Stage 1:** Fully connected network, all abilities are 0.6.
- **Characteristic**: Uniform distribution of capabilities



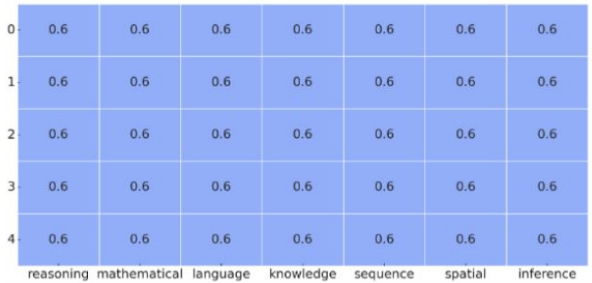
Intermediate State

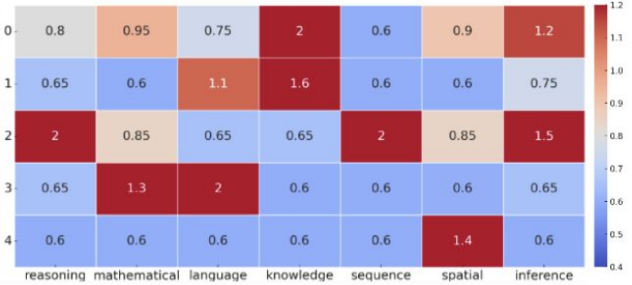
- **Stage 2:** Beginning of evolution, partial connections and variation in similarities
- **Characteristic:** Initial specialization and capability differentiation

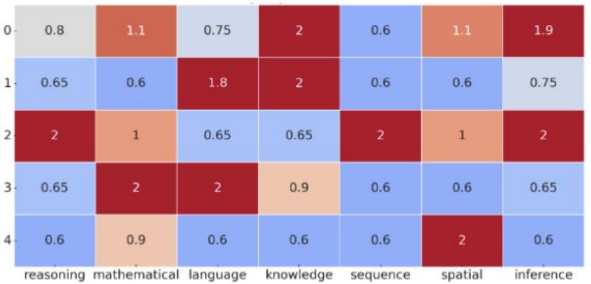


End

- **Stage 3:** More obvious specialization, increasing difference in agent similarities
- **Characteristic**: High specialization and dynamic adaptation







Building 17 Agentic AI Patterns and Their Role

- TBD

ソースコード

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L13]

- 状態グラフの状態（SupervisorState(TypedDict)）を定義する。
  - TypedDictはtypingライブラリに含まれるモジュールで、キーに格納されるデータの型をコード上で明示することが可能になる。[解説]
  - 状態の名称が SupervisorState となっているが、実際には他の専門エージェント（Researcher, Writer, etc）とも共有されているため注意。
  - messages: Annotated[list[BaseMessage], operator.add]
    - 会話履歴を表す。
    - Annotated[T, x] はtypingライブラリに含まれる型で、型Tそのものに対して任意のメタデータXを持たせることが可能になる。[解説]
    - つまり、messagesの型はlist[BaseMessage]で、そのメタデータは operator.addになる。

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L13]

- next_agent: str
  - 次に動作するべきエージェントを表す。
- task_complete: bool
  - タスクが完了したかどうかのフラグを表す。
- final_response: str
  - タスクが完了した場合の、ユーザーに対する最終応答を表す。

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L24]

- Supervisorが決定できる内容(RouteDecision(BaseModel))を定義する。
  - BaseModelはpydanticライブラリに含まれるモジュールで、出力データの検証や型ヒントを提供する。
  - next: Literal["researcher", "writer", "critic", "FINISH"] = Field(description="Which agent to call next, or FINISH if done")
    - 次に呼び出すべきエージェントの種類を表す。
    - Literalはtypingライブラリに含まれる型で、取りうる型を制限することが可能になる。nextは指定された4つの文字列(researcher, writer, critic, FINISH)のみを受け付ける。[解説]
    - Fieldはpydanticライブラリに含まれる関数で、BaseModel型のより詳細な検証ルールを設定することが可能になる。[解説]

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L24]

- reasoning: str = Field(description="Why this agent was chosen")
  - エージェントを選択した理由を表す。

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L34]

- SupervisorにいるLLMの出力を制限する。
  - supervisor_llm = llm.with_structured_output(RouteDecision)
    - .with_structured_output は langchain-coreライブラリ に含まれるメソッドで、引数で指定されたスキーマに基づいて、LLMの出力を構造化することが可能である。[公式Docs][解説]
    - 構造化出力の手法としては、以下の2種類がある。[公式Docs]
      ① .with_structured_outputメソッド
      ② create_agent関数の引数 response_format の指定
    - SupervisorにいるLLMの出力を、RouteDecisionで定義した型（next, reasoning）のみに制限している。

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L37-L70]

- SupervisorにいるLLMのシステムプロンプトを定義し、推論させている。
  - def researcher(state: SupervisorState) -> dict:
    - StateとしてSupervisorStateを与えている。
  - system_prompt = """You are a supervisor managing a team of specialists:
    1. researcher - Gathers information and facts … (省略)
    - システムプロンプトで与えている情報としては、以下の通り。
      ① 役割（あなたはSupervisorとしてチームを纏める必要がある）
      ② 配下にいる専門エージェント（researcher, writer, critic）の役割
      ③ ワークフロー（呼び出すエージェントの順序、etc）

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L37-L70]

- SupervisorにいるLLMのシステムプロンプトを定義し、推論させている。
  - messages = [SystemMessage(content=system_prompt)] + state["messages"]
    - 会話履歴（state["messages"]）を加えて、最終的なシステムプロンプトとしている。
  - decision = supervisor_llm.invoke(messages)
    - .invokeメソッドによりLLMが推論している。
  - return { "next_agent": decision.next, "messages": [AIMessage(content=f"[Supervisor] Routing to {decision.next}: {decision.reasoning}")], }
    - LLMが生成した次の遷移先（decision.next）を、状態グラフのState（state["next_agent"]）に格納している。（.with_structured_outputメソッドにより、decisionはRouteDecision型に構造化されている）

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L73-L94]

- ResearcherにいるLLMのシステムプロンプトを定義し、推論させている。
  - def researcher(state: SupervisorState) -> dict:
    - Supervisorと同じStateをResearcherにも与えている。(ここは参考にするべきではない。与えるStateは選別するべきだと考える)
  - system = """You are a research specialist. Your job: - Gather relevant facts and information - Be thorough but concise - Cite sources when possible - Focus on what's most useful for the task"""
    - システムプロンプトで与えている情報としては、以下の通り。
      ① 役割（あなたはResearcherです）
      ② 仕事内容と注意点

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L73-L94]

- CriticにいるLLMのシステムプロンプトを定義し、推論させている。
  - def critic(state: SupervisorState) -> dict:
    - Supervisorと同じStateをCriticにも与えている。(ここは参考にするべきではない。与えるStateは選別するべきだと考える)
  - system = """You are a quality critic. Your job: - Review the content objectively - Provide specific, actionable feedback - If the work is good, say "APPROVED" and explain why - If it needs work, explain exactly what to improve"""
    - システムプロンプトで与えている情報としては、以下の通り。
      ① 役割(あなたはCriticです)
      ② 仕事内容と注意点

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L153-L157]

- Routerで行う処理を定義している。
  - if state.get("task_complete"): return "finalize" return state["next_agent"]
    - task_completeがTrueの場合には、Finalizeノードに遷移する。
      それ以外の場合は、state["next_agent"]ノード（＝LLMが生成した次の遷移先（supervisor関数のreturnを参照））に遷移する。
    - .add_conditional_edges メソッドの第二引数で、Routerの処理を規定する必要がある。

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L160-196]

- 状態グラフを構築している。
  - graph = StateGraph(SupervisorState)
    - StateGraph は langgraphライブラリに含まれるクラス。公式Docsの説明を見ると「A graph whose nodes communicate by reading and writing to a shared state.」とある。
    - グラフ構造は状態として SupervisorState を保持している。
  - graph.add_node("supervisor", supervisor)
    - .add_node(node, action, etc) は langgraphライブラリに含まれるメソッドで、nodeにノード名[str]を、actionに 関数 or Runnbaleオブジェクト を指定する。[公式Docs]

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L160-196]

- graph.add_conditional_edges("supervisor", route_to_agent, { "researcher": "researcher", "writer": "writer", "critic": "critic", "finalize": "finalize", },)
  - .add_conditional_edges(source, path, path_map) は langgraphライブラリに含まれるメソッドで、sourceに条件付エッジを始めるノード名[str]、pathに次に遷移するノードを決めるための関数、path_mapに遷移できるノードの一覧[dict] を指定する。[公式Docs]
- return graph.compile()
  - .compileメソッドは langgraph に含まれるメソッドで、コンパイルされた実行可能なグラフ（CompiledStateGraph）を生成する。[公式Docs]
  - CompiledStateGraph は Pregel を継承しているため、継承元の .invoke メソッドなどを使用できる。[公式Docs][解説]

pdichone/langgraph-multi-agent-supervisor

[コードの内容 - L199-L235]

- 本プログラムの実行部分
  - agent = build_multi_agent_system()
    - build_multi_agent_system の return（＝CompiledStateGraph）が agent に格納される。
  - initial_state = { "messages": [HumanMessage(content="Write a short blog post about the benefits of AI in healthcare")], "next_agent": "", "task_complete": False, "final_response": "", }
    - 状態グラフ(SupervisorState)の初期状態を定義している。

GPUプログラミング

OpenACC ディレクティブによるプログラミング

[1章 - CPUとアクセラレータの違いは？]

「CPU」と「アクセラレータ(GPU)」の構造的な違いを纏めたよい資料がある。PGI の Michael Wolfe の資料(*1) をもとに、性能に関与する構造上の違いを以下の表に表した。「アクセラレータ(GPU)」は、その動作周波数はきわめて低く、スカラ処理する際のベース性能はCPUに比べて低いことが分かる。その代わりに、大量のコアプロセッサを用いて、SIMD(ベクトル) 処理と MIMD 並列処理の両方を行って性能を稼ぐ方法をとっている。プロセッサの構成も CPU に比べて単純であり高度な分岐予測等は行わず in-order で実行処理を行い、命令処理のストールを隠蔽するために、多重のマルチスレッディング実行ができるようになっている。すなわち、こうしたアクセラレータ上では、SIMD(ベクトル)処理と並列処理が可能なプログラムが、その本来有している高い性能を享受することが出来る。

CPU		アクセラレータ(GPU)
基本的な性能のベースとなる動作周波数		
高速クロック周波数 (2.5～3.5GHz)	>	低速クロック周波数 (0.8～1.0GHz)
クロック当たりの命令実行を増やす工夫		
パイプライン (深い)		パイプライン(浅い)
Multiscalar (3-4)	>	Multiscalar (1-2)
SIMD命令当たりの処理数 (4-16)	<	SIMD命令当たりの処理数 **(16-64)**
プロセッサコア数 (6-12)	<	プロセッサコア数 **(15-32)**
命令実行のストール(wait)を少なくする工夫		
大きなキャッシュメモリ搭載	>	小さなキャッシュメモリ搭載
高度な分岐予測	>	-
Out-of-order execution	>	In-of-order execution
マルチスレッディング (2-4)	<	マルチスレッディング **(15-32)** ; 大量のレジスタ数

GPU プログラミングを行う上で、NVIDIA GPU の構造、特に、スレッドの処理単位やベクトル処理の方法に関して書いた拙著「NVIDIA GPU の構造と CUDA スレッディングモデル」は、2010 年に書いたものであるが、GPUの基本的な構造はあまり変わらないため、一読していただくと今後の参考になるかもしれない。

・スカラ処理とは、浮動小数点演算処理を逐次的に行う処理方式のこと。

OpenACC ディレクティブによるプログラミング

[1章 - アクセラレータ用のプログラミングモデル]

このアーキテクチャで特徴的なことは、以下の点である。

1. CPUのメモリ空間とアクセラレータ・デバイスのメモリ空間が異なること
2. 一般的なメモリ帯域幅に比べて低速な PCI バスで接続されていること

このような構成の上でプログラミングを行うことを想定してみよう。ある重い処理をアクセラレータ・デバイス（以下、**「デバイス」とも言う。**）側で処理をしようとした場合、CPU 側に存在する必要なデータをデバイス側にコピーし、処理が終わった後に、CPU 側に戻すと言うハンドリングが必要となる。また、これと関連して、両者のメモリ空間が異なるため、一つのプログラム内で単一アドレス空間でのメモリアクセスが出来ない（あるいは同じ変数名が使用できない）ことにより、プログラミングの難易度が増すと言った欠点がある。

こうしたプログラミング上の問題を出来る限り吸収し、例えば、NVIDIA のネイティブな CUDA 言語よりも簡単に、さらに従来の高級言語によるプログラムの中にコンパイラ・ディレクティブを挿入するだけで、アクセラレータ(デバイス）コードを生成出来るようにしたものが OpenACC プログラミング仕様である。現在、一番成功したと言われる共有メモリアーキテクチャに対する「並列プログラミングモデル」である OpenMP と同じ手法を踏襲したものと考えて良い。OpenACC プログラミングモデルの原型は、PGI が 2008年に提唱した **PGI Accelerator Programming Model** であり、2009年11月に PGIアクセラレータコンパイラとしてその実装形を市場に提供した。その後、2011年11月に、PGI は、NVIDIA社、Cray社、CAPS社と共に、OpenACC API 1.0 仕様書を定義し、現在、OpenACC 対応のコンパイラを提供している。さらに、2013 年 7 月に **OpenACC 2.0** の標準仕様を発表し、その実装コンパイラが 2013 年末から 2014 年の始めに各社から提供されることとなっている。OpenACC は、下図のように 「CPU+アクセラレータ」のハイブリッド構成に対するプログラミングモデルであり、**CPU処理をオフロードしデバイス側で処理するためのコード開発**を支援するディレクティブ群と理解して良い。また、もう一つの仕様として2013年7月に、New **OpenMP 4.0** の中の機能としてアクセラレータデバイス構文（target 構文 directive）が確定され、同様なコンセプトで今後の標準化を見据えている。

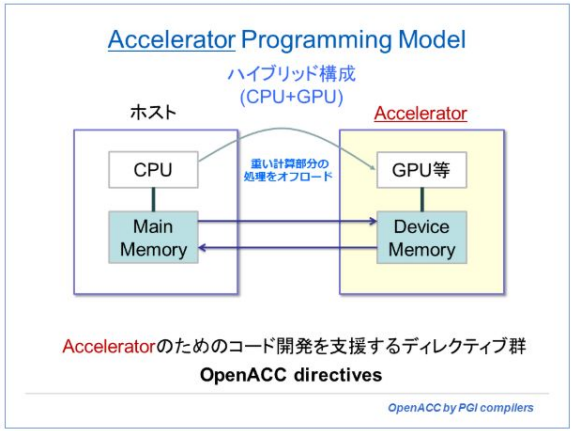
・従来の課題：「CPU+アクセラレータ」のハイブリット方式では、メモリ空間がそれぞれ異なるため、プログラム内で単一アドレス空間でのメモリアクセスができないため、プログラミングが複雑化するという欠点があった。

・メモリアクセスとは、記憶領域（RAM等）に対して、データの読み書きや参照を行う操作のこと。

OpenACC ディレクティブによるプログラミング

[1章 - アクセラレータ用のプログラミングモデル]

こうしたプログラミング上の問題を出来る限り吸収し、例えば、NVIDIA のネイティブな CUDA 言語よりも簡単に、さらに従来の高級言語によるプログラムの中にコンパイラ・ディレクティブを挿入するだけで、アクセラレータ(デバイス) コードを生成出来るようにしたものが OpenACC プログラミング仕様である。現在、一番成功したと言われる共有メモリアーキテクチャに対する「並列プログラミングモデル」である OpenMP と同じ手法を踏襲したものと考えて良い。OpenACC プログラミングモデルの原型は、PGI が 2008年に提唱した **PGI Accelerator Programming Model** であり、2009年11月に PGIアクセラレータコンパイラとしてその実装形を市場に提供した。その後、2011年11月に、PGI は、NVIDIA社、Cray社、CAPS社と共に、OpenACC API 1.0 仕様書を定義し、現在、OpenACC 対応のコンパイラを提供している。さらに、2013 年 7 月に **OpenACC 2.0** の標準仕様を発表し、その実装コンパイラが 2013 年末から 2014 年の始めに各社から提供されることとなっている。OpenACC は、下図のように「CPU+アクセラレータ」のハイブリッド構成に対するプログラミングモデルであり、**CPU処理をオフロードしデバイス側で処理するためのコード開発**を支援するディレクティブ群と理解して良い。また、もう一つの仕様として2013年7月に、New **OpenMP 4.0** の中の機能として**アクセラレータデバイス構文（target 構文 directive）**が確定され、同様なコンセプトで今後の標準化を見据えている。



OpenACC ディレクティブによるプログラミング

[1章 - OpenMP 4.0 target 構文と OpenACC 規約]

筆者の考えでは、OpenMP 4.0 の target 構文でアクセラレータ・プログラミングが可能な内容と OpenACC 2.0 レベルで可能なことを比べた場合、OpenMP 4.0 はその機能自由度の観点でまだ見劣りする点があり、アクセラレータデバイスに関する標準規約としては、3 年余り先じている **OpenACC がデファクト・スタンダード**となるのではないかと思っている。これに関してもう一つ重要な視点は、現在市場に存在するアクセラレータデバイス（NVIDIA、AMD、Intel、その他DSP、FPGA等）の中で、OpenMP 4.0（過去の3.1機能仕様も当然含む）の規約を実装したソフトウェア環境を有してプログラミングできるデバイスは、インテル Xeon Phi しか存在しない。また、**インテル社の** Xeon Phi（MICアーキテクチャ）用のもう一つの**オフロード・プログラミングモデル**は、独自のディレクティブ仕様であり、インテル社以外の「デバイス」上では利用出来ない。さらに、インテル社がこのプログラミングモデルの後継としている OpenMP 4.0 の target 構文は、複数のデバイス間のプログラム互換性を推進するためのオープンな規約であるにも拘わらず、皮肉にもこの規約で動作するアクセラレータ・デバイスは、 現在、Xeon Phi 以外存在しないと言う情況にある。仮に、インテル社の動作環境で Phi ではなくマルチコアCPU を target として動作させようとしよう。この場合、プログラム上に Phi デバイス用の「OpenMP target構文」のオフロード・ディレクティブが記述されている場合、どのように動作するのだろうか？完全に無視されてしまうかどうかは分からないが、これだと、同じ OpenMP ディレクティブのソースコードでありながら、「マルチコアCPU」上では、並列化動作できないと言う不自然さが生じる。一つのソースコードでのプラットフォーム互換性に問題が出る。これが、OpenMP 4.0 の正しい動作かどうかは調べていないが、CPUとデバイスの両者で多くの feasibility の知見とそれを改善してゆく時間が必要なのではないかと感じざるを得ない。

OpenMP 4.0 に準拠するということは、それ以前の OpenMP 3.1 までのマルチコアベースの並列化機能も取り込む必要がある。x86 アーキテクチャを有する Intel Xeon Phi の場合は、現在のインテル社の OpenMP 環境をそのまま踏襲できるため、これは可能である。しかし、他社のデバイスに関しては、そもそも OpenMP エンジンを有していないため、当該デバイス用の OpenMP 3.1 機能を含めた 4.0 の環境の新たに開発を行うことは、あまり合理的な話ではない。となると、インテル社以外のデバイスで、OpenMP 4.0 に対応することはかなり敷居が高いと言えよう。一方、インテル社自体も Xeon Phi のようなコプロセッサに対する並列ソフトウェア環境が OpenMP 4.0 ベースのままで踏襲していくと、アクセラレータ用のソフトウェア開発環境においては、今後益々、他デバイスとのプログラム互換性という観点で孤立感を高めてしまう可能性もある。OpenACC 2.0 では、すでに Xeon Phi も含めたマルチベンダーデバイスを利用出来る仕様策定がなされており、PGI では 2014 年にこの 3 社のデバイス用の OpenACC コンパイラを市場提供する。OpenACC の使い勝手の向上や、ネイティブな並列言語 CUDA C/C++、CUDA Fortran 等の言語体系とのインターオペラビリティの向上、さらに異なるマルチベンダーデバイス上で OpenACC 実装が利用出来ることになると、OpenACC の利用にも弾みが付く可能性がある。特に、OpenACC の特長は、「プログラム互換性」だけでなく、複数のデバイス間でも、それぞれの特性に応じたベスト・パフォーマンスが得られること、すなわち「性能互換性」を志向したプログラミング・モデルでもあることだ。そのためにユーザが指定可能な directives と clauses が用意されている。なお、2013 年 11 月に米国のスーパーコンピュータ「Titan Cray XK7」を保有している Oak Ridge Leadership Computing Facility は、**GCC (Linux) の OpenACC 2.0 への対応を行うための取り組みを進めている**ことを発表した。オープンソースのコンパイラも OpenACC への実装が始まりつつある。

・OpenACCとOpenMPの違いとしては以下の通り：
①OpenACCは異なるマルチベンダーデバイス上で実装可能である。（OpenMPは、ver4.0時点ではインテル社のXeon Phiでしか実装できないとのこと）
②OpenACCはネイティブな並列言語（C, C++, Fortran等）の言語体系との互換性がある。

OpenACC ディレクティブによるプログラミング

[1章 - オフロード形式のプログラミングモデルの合理性]

一般に、アクセラレータは「動作クロック周波数」が低いため、この上で「スカラ部分の実行」を行うことは、大きくプログラム全体性能の低下をもたらす。こうしたスカラ実行部分は高速な CPU 側で実行し、アクセラレータ上では計算 intensive な部分をベクトル・並列実行で集中的に行うと言う構図が、現状のアクセラレータ・アーキテクチャを考えた場合、合理的な使い方となる。これが、すなわち、これが**「オフロード形式」の実行形態**なのである。アクセラレータのプログラミングモデルが「オフロード形式」にならざるを得ない一つの理由は、一般的なアクセラレータの「動作クロック周波数」が低いためであることに因るものである。

・CPUとGPUの使い方の切り分けイメージ：
　・CPU：スカラ計算を実施
　・アクセラレータ（GPU）：ベクトル計算（C++でいう、std::vectorの計算？）や並列計算を実施

OpenACC ディレクティブによるプログラミング

[1章 - SIMDベクトル演算とアムダールの法則]

現在、市場に流通する三社のアクセラレータは、共に **wide-vector operation** が可能である。すなわち wide-SIMD 処理できる。但し、各社のデバイスのその動作方法は異なる。デバイス上の SIMD 処理は、gather/scatter 等のマスキングを使い「条件」を含んだベクトルを同時処理することが出来ないものもあるため、厳密な定義でのベクトル演算とは言えないが、ここでは広義のベクトル処理として扱う。Intel Xeon Phi の場合は、単精度レベルで 16-wide のベクトル長、NVIDIA Kepler は、32-wide(warp)、AMD Radeon は、64-wide(wavefront) 単位の SIMD(ベクトル)処理が可能である。プログラム的見地では最内側のループを対象としてベクトル・SIMD 処理の基本単位を作り、その外側ループ構造で MIMD 並列単位を作ることで、数多くの並列実行実体を生成して、ハードウェアの並列演算コアとのマッピングを行い、ベクトルならびに並列実行にて高速化を図る。オフロード形式のプログラミングモデルでは、必ず、こうしたベクトル化かつ並列化可能なループ構成をオフロードの対象としてプログラミングを行う。一方、Xeon Phi の**ネイティブモード**の場合においても、ベクトル処理を有効に活用しなければ、理想的な高速化ができない。アクセラレータでは、単に「並列処理」の高速化だけではなく、その基本実行主体が「ベクトル処理」されていることによって高速化が達成されるのである。従来の CPU においては、最近の AVX 機構でも高々 256-bit wide (単精度で 8、倍精度で 4 のベクトル長) の SIMD が提供されているものの、こうした短いベクトル長であったため、プログラムの「ベクトル化率」が全体プログラムの性能にどの程度影響を及ぼすかと言う議論はあまりなかった。実は、20~30年前にスーパーコンピュータが「ベクトル機」の全盛時代であった時は、プログラムの「ベクトル化率」が性能を左右する大きな要素であった。ベクトル機で性能高速化する際の最も重要な点は、全体プログラムの中でスカラ演算部分をどれだけ排除するかと言う点、すなわちベクトル化率を出来るだけ 99% に近づけると言う最適化方針が存在した。アクセラレータにおいても、wide-vector operation が出来るようになったため、ベクトル演算効果を阻害しないプログラム作りが必要となる。もちろん、オフロード対象となるループ部分は、データ依存性がないため、本来ベクトル化も並列化も可能であることが普通であるが、「ベクトル演算効果を阻害しない」というところに意義がある。効果を最大限に高めるには、例えば、stride-1 のメモリアクセスや if 分岐の論理判断の合理的処理等の方策がチューニングにおいては必要となると言うことである。

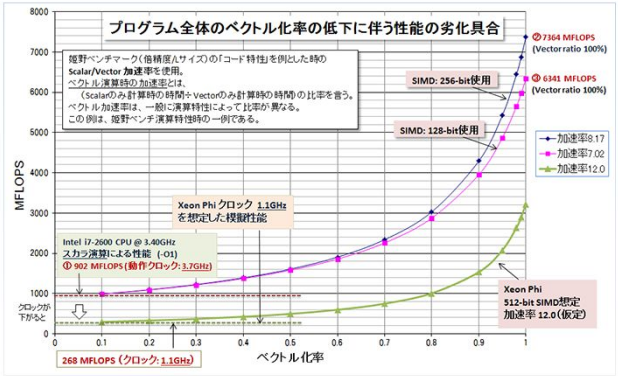
プログラム全体の何%が並列化可能かと言う指標は「並列化率」と言うが、この割合により並列効果が変化する。並列化しても問題の大きさが変化しないという前提の下、これをモデル化したのがアムダールの法則である。このアムダールの法則を使って、ベクトルプロセッサを対象としたモデル化もできる。プログラム全体のベクトル化可能な割合であるベクトル化率の他に、もう一つのパラメータとして、スカラ実行した場合の時間とベクトル実行した場合の時間の比率（これを「ベクトル加速率」という。）をモデル化パラメータとすれば、全体のプログラムのベクトル化率割合に応じた性能のシミュレートが出来る。但し、ベクトル加速率は、プログラムの演算特性（計算密度や演算命令の種類）によって変化するため、予め何らかの方法で仮定することが必要となる。

・並列化率については、以下URLのスライド（P.7, P.8）にて詳細の説明あり：
https://www.hpc.cmc.osaka-u.ac.jp/wp-content/uploads/2022/04/20220616.pdf

OpenACC ディレクティブによるプログラミング

[1章 - SIMDベクトル演算とアムダールの法則]

以下の図は、全体のプログラムの実行時間がベクトル化されたループだけで構成される「姫野ベンチマーク」を例に、Xeon Phi の 61コアある中の 1コア内（512-bit vector 命令）で行われるベクトル演算の性能を模擬してみたものである。仮定に基づいているため、個々の模擬(絶対)性能を議論するのではなく、ベクトル化率に応じた性能曲線の傾向や絶対性能のベースとなる「動作クロック周波数」の違いによりベクトル性能も決まること、ベクトル化による性能効果はベクトル化率 95% 以上であれば加速率が高くなることなどを理解して欲しい。前項の話題でも触れたが、**Xeion Phi のネイティブモードで実行**する際は、全体のプログラムとして、このベクトル演算の恩恵を得ることが重要であるため、プログラム全体のベクトル化率が低いコードの場合、ネイティブモードで実行しても大きな性能の享受を期待できないことが予想できる。繰り返しになるが、**Xeion Phi のネイティブモードを利用する際**は、ベクトル化・並列化が可能な実行割合が 90% を超えるようなプログラムでないと、それ相当の性能を期待できないように思う。**「ネイティブでコンパイル」**できると言うと、どのようなプログラムでも Xeon Phi の高速性の恩恵を受けることができると言った誤解を生んでいるようだが、スカラ計算部の比率が高いプログラムではその高速化は難しいだろう。ということを総合すると、筆者の意見は、Xeon Phi も含めて一般的なアクセラレータの性能を最大限引き出すためのプログラミングモデルは、「オフロード型のモデル」が今のところ最適であろうと思っている。この理由は、ベクトル化による性能効果を期待できないスカラ部分は高速な CPU に処理を任すことによって、CPU とアクセラレータのそれぞれの良さを活用できるからである。但し、その際に問題となるのは、CPU とアクセラレータ間のデータ転送時間を最小化すると言う課題だけは残る。



・CPUとGPUの使い方の切り分けイメージ（再掲）:
　・CPU：スカラ計算を実施
　・アクセラレータ（GPU）：ベクトル計算（C++でいう、std::vectorの計算？）や並列計算を実施

・CPUとアクセラレータ間のデータ転送にかかる時間を最小化する必要がある点については、意識する必要がある。

OpenACC ディレクティブによるプログラミング

[1章 - プログラムの中でオフロードする対象部分とは]

Fortran/C/C++ 言語で記述されたプログラムの中で、時間が掛かる処理部分をオフロードして、アクセラレータ側で処理することが一般的である。これは、多くの場合、「Do / for ループ」である。要は、**ループ構造を有する繰り返し部分を「並列処理で高速化する」こと**をアクセラレータ上で実現する。プログラム内にこうしたループ構造が少ない場合、並列化が出来ないあるいは、並列効果が現れない場合が多いため、そもそもアクセラレータによる高速化の対象外となる。すなわち、プログラム内で「時間を消費するループ構造」が存在すれば、アクセラレータによる高速化の対象とすることができる。また、「並列処理で高速化」を行うと言うことに関しては、マルチコア環境下のOpenMP による場合と同じ構図となる。従って、OpenMP を適用して並列化出来ているプログラムは、OpenACC適用の対象となる。但し、享受できる並列性能に関しては OpanMP の場合とは異なるチューニング要素があるため、アクセラレータで常に性能加速率が良くなるとは言えない。最後に大事な点は、並列対象となるループでは、並列実行における阻害要因がない（並列依存性がない）ことが求められる。並列依存性が存在するループ構造の場合は、これをなくすためのプログラムの変更が必要とされ、これが不可能な場合は、そもそも「並列化」による高速化はできないことは言うまでもない。

・ループ構造を有する繰り返し部分（forループ等）が、オフロード対象である。

・OpenACCにはチューニング要素がある点、及び並列実行への移行に伴う阻害要因がないループ構造を採用する必要がある点については、意識する必要がある

OpenACC ディレクティブによるプログラミング

[1章 - プログラムの中でオフロードする対象部分とは]

OpenACC Standard について簡単に説明する。これは、Accelerators（デバイス）用のプログラミング API の標準仕様である。具体的に言えば、Fortran, C/C++ 言語上で指定するコンパイラ・ディレクティブの仕様とこの機能を支援するための API ルーチン群を言う。特に、CPUホストからアクセラレータ側に**処理をオフロードするための目的**に策定されたものである。

- ユーザサイドでは、開発者がアクセラレータで実行するコード部分をディレクティブで指定する(コンパイラに対して、ヒントを与える)
- OpenACC コンパイラは、ホスト側の処理をデバイス側にオフロードするコードを生成し、また、ホストと デバイス間のデータ転送コードの生成を行うと言った機能を果たす

OpenACC の利点、特長は以下の点にある。

- **可搬性(ポータビリティ)：**ホストOS, ホストCPUs, アクセラレータ、コンパイラ等、異なるプラットフォームに変わっても「性能」の可搬性を提供
- **ハイレベル／高級言語：**基本的に、CUDA, OpenCL or 低レベル GPU プログラミング等の難しい言語知識の必要がなく、現在のソースコードの変更を必要としない
- **高生産性：**迅速な開発ステップと段階的に行える開発
- **ヘテロジニアスなプログラム：**シングル・ソース・ベースで維持管理可能(CPU用、デバイス用と分ける必要なし)

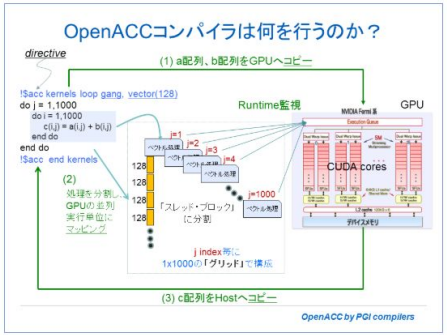
OpenACC ディレクティブによるプログラミング

[1章 - OpenACC コンパイラが行うこと]

プログラム内に、「ディレクティブ」でヒントを与えると、コンパイラは、次のような機能を組み込んだホスト側コードとデバイス側コードを自動生成する。コンパイラは、これら両者の「実行コード」リンクし、一つの実行形式のバイナリファイルとして生成する。

- デバイスをイニシャライズ
- ホスト(CPU)とデバイス間の「データ」と「プログラム」の転送の管理
- ホスト(CPU)とアクセラレータ間の「データ」を監視
- ホスト(CPU)とアクセラレータ間の「仕事」を管理
- 「仕事」を**並列に分割**して、デバイス・ハードウェアの並列構造にマッピング
- さらに、性能最適化を試みる

以下の図は、具体的に Fortran のループ処理を例にとって、OpenACCコンパイラはどのような処理を行うコードを作成するか模式的に示した。ここでは、GPU をアクセラレータとしている。ネスト・ループの内側ループを SIMD/Vector 処理を行うために、適当な長さ単位(チャンク) に分割する。この単位でトータル 1000 個の要素を計算処理する。また、外側ループは、内側ループで行われる SIMD/Vector 処理のブロックを 1000 個用意し、これを並行に動作させるコードを作り上げる。1000 要素のベクトル処理を行うブロック 1000 個、GPU 上のタスクキューに並べ、有効な GPU コアの中に当該ブロックを並行にタスクとして投入する。ここまでが、オフロードされた仕事の処理の方法を説明したものであるが、もちろん、その前に、GPU側で必要とするデーターの転送を行う。同様に計算結果となるデータも GPU から CPU 側に戻す処理も行う。こうしたデータ転送に係わる処理命令もコンパイラは自動的に作成する。以上が OpenACC コンパイラが作り上げるコードの概要である。

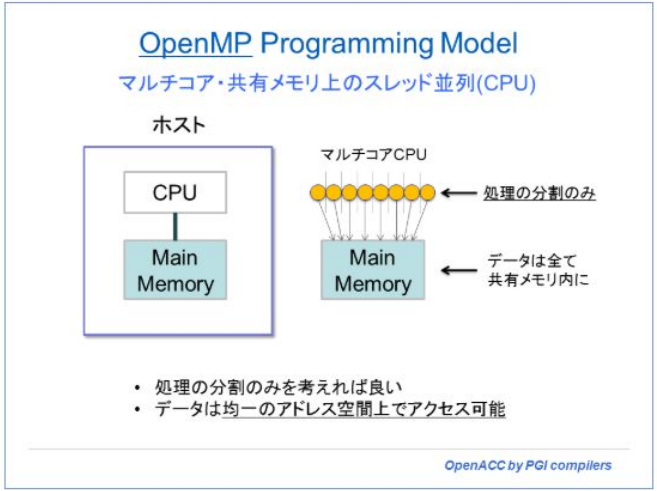


・詳細は図を参照。

OpenACC ディレクティブによるプログラミング

[2章 - OpenMPプログラミングモデル]

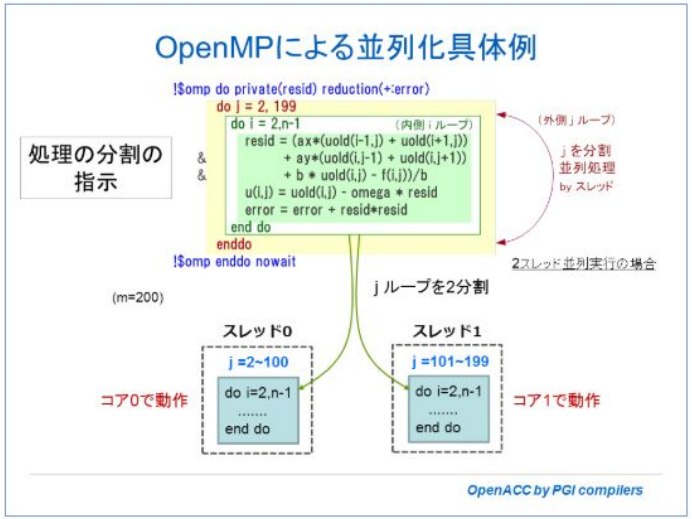
OpenMP プログラミングモデルは、共有メモリ構成を有するマルチコアCPU上で、マルチスレッド並列化を行うためのプログラミング手法である。ユーザは、プログラム内の並列化可能なループに対して、コンパイラ・ディレクティブで指示することにより、スレッド並列コードを生成できる。各コア上で分割した処理をスレッド実行することによって並列効果を得る。メモリ空間は共有できるため、プログラマは「処理の分割の指示」を行うだけで済む。OpenMP は、並列化指示のためのディレクティブ仕様とこの機能を支援するための API ルーチン群を言う。下図は、OpenMPのスレッド実行の概念図である。



OpenACC ディレクティブによるプログラミング

[2章 - OpenMPプログラミングモデル]

具体的にディレクティブを挿入した場合の例を示そう。下図は、Fortran プログラムのループに対して、並列化の指示を行っている様子を表したものである。j ループを分割し、異なるスレッドで並列実行する形を示している。なお、OpenMP の場合、配列等のデータはメモリ共有を想定しているため、データアドレス空間の分割等の作業は必要ない。



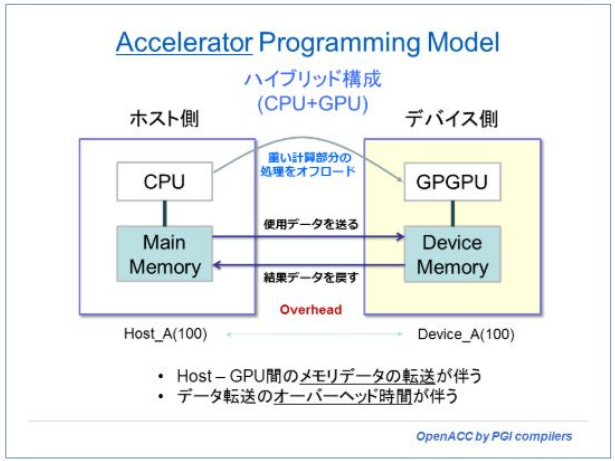
・本図が分かりやすい。ディレクティブを付加した外側のループ構造が、並列化されていることが分かる。

OpenACC ディレクティブによるプログラミング

[2章 - OpenACC (アクセラレータ) プログラミングモデル]

OpenACC プログラミングモデルの動作概念図を以下に示す。この図では **Accelerator Programming Model** と称しているが、OpenACC のモデルと同じと考えて良い。ホスト側 CPU で実行する一部の処理をオフロードして、デバイス側で処理を行う場合の実行モデルである。このページの説明では、デバイスを GPU とした場合を例とした図を描いている。「GPGPU」とは、General Purpose GPU の意味である。

プログラムの中で、時間を消費し並列化可能とするループ部分を切り出して、デバイス側で（オフロード）処理を行う。そのためには、当該ループプログラム部分のデバイス用コードを作成しなければならない。例えば、デバイスが NVIDIA 社の GPU であれば、CUDA 用のバイナリを生成することが必要となる。また他のデバイスであれば、当該デバイス用の実行バイナリを作成することになる。コンパイラは CPU 側の実行形式とデバイス側の実行形式を作成し、これら二つの実行形式バイナリをバインドして、一つの実行ファイルとして生成する。



・本図は、ホスト側（CPU）で実行する処理の一部をオフロードして、デバイス側（本例ではGPU）で処理を行う場合の模式図である。

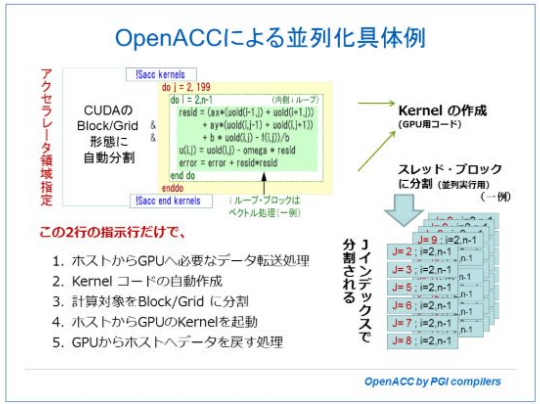
・現代コンピュータは、扱う情報を全てビット形式（つまり0or1の2進数）で管理している。つまり、厳密にはコンピュータ上で扱うファイルは全て「バイナリファイル」である。この「バイナリファイル」は、デバイスごとに形式がことなる。

OpenACC ディレクティブによるプログラミング

[2章 - OpenACC (アクセラレータ) プログラミングモデル]

上記の通りプログラムによる処理部分はCPU部とデバイス用に分割される。必要とするデータはどうであろうか？上図のような二つのメモリ空間を配するハイブリッド構成で動作させるためには、デバイス側に対して計算処理に必要なデータをコピーしなければならない。また、処理が終了した後、CPU側へデータを戻すことが必要となる。必要とするデータを転送する部分は、OpenACC コンパイラが担い、基本的に自動的にそのコードを作成する。あるいは、ユーザが明示的にディレクティブを指定してデータのコピーを指示することも出来る。これらに必要な時間は、全て**オーバーヘッド**となる。こうしたオーバーヘッドが加算されたとしても、全体の性能(経過時間) として CPU のみで実行する時間よりもメリットがあれば、アクセラレータを使用する意味が出てくる。

以下の例は、上記 OpenMP の例と同じ Fortran プログラムのループに対して、OpenACC ディレクティブで指示した様子を表したものである。OpenMP の場合と同じように j ループを分割し、それぞれの単位が並列実行主体(スレッド・ブロック) となる。このスレッド・ブロックがアクセラレータのコア上で独立に処理される形態をとる。また、OpenACC モデルの場合、内側の i ループ自体も並列に分割して並列多重度を高めて実行する場合が多い。なお、OpenACC の場合、デバイス上で使用する配列等データの転送は、コンパイラによって自動的にそのコードが生成される。



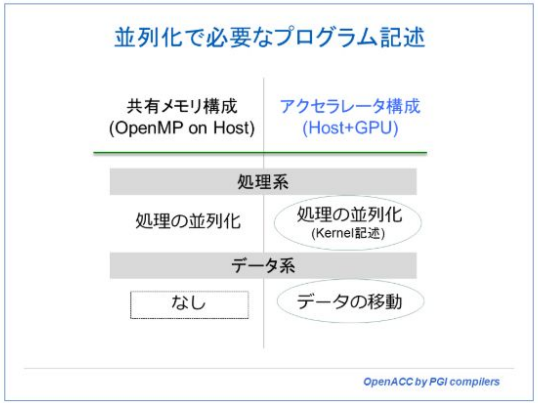
・本図も分かりやすい。

OpenACC ディレクティブによるプログラミング

[2章 - OpenMPとOpenACC (アクセラレータ) プログラミングモデルの違い]

OpenMP と OpenACC のプログラム記述上の違いを説明しよう。ディレクティブを使用してプログラムに指示するべきものとして、「処理系の指示」と「データに関する指示」の二つに分けられる。処理系に関しては二つのプログラミングモデル共に、「処理の並列化の指示」を行う必要がある。しかし、データ系に関してアクセラレータ（OpenACC）モデルでは、データのコピーに関する処理を行う必要がある。デフォルトでは暗黙にコンパイラがデータコピーのためのコードを生成するが、最適な性能を得るためには、データコピーを制御するために明示的にディレクティブを挿入する必要が出てくる。OpenMP の場合は、共有メモリ空間を使用するため、こうしたデータコピー等の概念はそもそも存在しない。これが、二つのプログラミングモデルのディレクティブの記述における大きな違いと言える。逆に言えば、CPU とデバイス間のデータコピーに係わる時間（オーバーヘッド）は、OpenACC（アクセラレータ）プログラミングにおいて、当該性能を左右する大きなポイントとも言える。

プログラム開発の観点では、現在 OpenMP 化されているプログラムは、OpenACC に置き換えることは容易である。ループに対する並列化が可能であることがすでに分かっているため、OpenMP directives を OpenACC directives に変更すれば良い。そして、その際に行う追加作業は、最適なデータコピーとなるように OpenACC data directive を挿入し、データコピーの最適化を行うこととなる。



・OpenMPとOpenACCの違い：
　・OpenMPは、メモリ空間が共有されている前提での利用のため、データ転送についてはサポートしていない。
　・OpenACCは、メモリ空間が共有されないハイブリット構成（CPU+アクセラレータ）でも利用可能なため、データ転送についてもサポートしている。

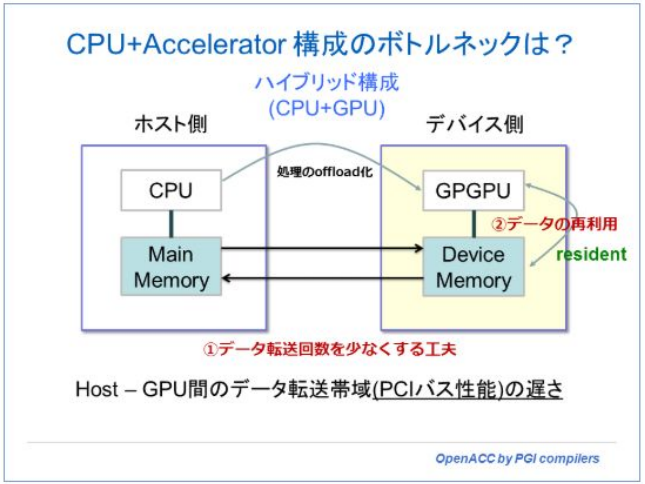
・上記から、ハイブリット構成の場合はOpenACCを利用することになる。

OpenACC ディレクティブによるプログラミング

[2章 - CPU+アクセラレータ構成における性能のボトルネック部分]

CPU+アクセラレータ（デバイス）構成において性能面で加わるオーバーヘッドは、CPU とデバイス間のデータ転送に伴う「時間」である。頻繁にデータ転送を行うような形になると、こうしたプログラミングモデルによる「性能効果」は望めず、CPU のみで実行した場合より時間が掛かる場合がある。これは、デバイスとの接続が PCI Express バス（Gen2 で 5～6GB/sec程度）を経由することによる遅延が原因であるが、それ以上に、データ転送の回数を最適化しないことによる問題の方が大きい。OpenACC を使用する場合には、OpenMP の場合とは異なり、以下に述べるデータの転送に係わることに注意をしてプログラミングする必要がある。

1. CPU～デバイス間のデータ転送回数を極力少なくする
2. 一度、デバイスに転送したデータを常駐させ、データの再利用を図るようにする



OpenACC ディレクティブによるプログラミング

[2章 - CPU+アクセラレータ構成のメモリモデル]

OpenACC 等のアクセラレータ・プログラミングモデルにおけるメモリに係わる論点を整理する。このプログラミングモデルでは、以下のようなメモリ・ハンドリングに関する特徴がある。以下では、アクセラレータ・デバイスのことを GPU として表記している。

- ホスト側 CPU のデータ空間とGPU 側のデータ空間の二つの空間が必要
- CPU と GPU 間のデータ転送が必要、これが overhead 時間となる
  - データコピーを明示的に制御するには、そのためのディレクティブが必要
  - データコピーの最小化を行う必要がある
- ホスト側CPUメモリとGPU側メモリ間の転送帯域(PCI帯域)に制約があるため、オフロードすべき対象を絞る必要がある
  - 性能加速性があるループ領域かどうかの判別が必要
  - ループ内の「計算密度」のレベルで加速性が変化することを認識する
- GPU側のメモリ容量に制約がある(現在、< 6GB)
  - 非常に大きな配列を伴う「コード領域」のオフロード化に制約
  - もし必要な場合、複数の GPU デバイス上の MPI 化が必要

